

GUIA DE ESTUDO

Modelação e Design (MeD)

Análise e Desenho Orientados a Objetos com UML

Licenciatura em Engenharia Informática

Toda a matéria + exercícios resolvidos

Casos de uso · Atividade · Domínio · Classes · Objetos · Sequência · DSS · Comunicação · GRASP · Padrões GoF · Componentes ·
Instalação · Processo de desenvolvimento

Índice

Como usar este guia

Este guia cobre os 14 blocos das aulas teóricas e está organizado para servir tanto para estudo de fundo como para revisão de véspera. Cada tema tem **conceitos essenciais**, **notação UML** e os **erros típicos**. No fim há uma secção de **exercícios resolvidos** construída a partir do formato real dos exames de MeD, e uma **checklist final**.

Estrutura recorrente do exame (formato “longo”, 2011–2018)

Os exames de época normal/recurso repetem quase sempre cinco grupos. Reconhecer o padrão poupa tempo:

1. **Grupo 1 – Casos de uso.** Dado um enunciado + figura: identificar casos de uso e atores; atualizar o diagrama (acrescentar um caso com «include» ou «extend»); comentar uma afirmação.
2. **Grupo 2 – Atividade → Sequência → Comunicação.** Converter um diagrama de atividade num diagrama de sequência (com as chamadas aos métodos indicados), depois no diagrama de comunicação equivalente; por vezes acrescentar pistas/partições; comentar uma afirmação.
3. **Grupo 3 – Classes → Objetos.** Dado um diagrama de classes conceptual, validar cenários desenhando o diagrama de objetos correspondente ou justificando porque não é possível (viola multiplicidades/restrições).
4. **Grupo 4 – Da análise ao design.** A partir de um requisito: escrever o fluxo de eventos do caso de uso, construir o modelo do domínio (análise linguística), o diagrama de sequência do sistema (DSS) e desenhar a solução para uma mensagem aplicando GRASP/MVC.
5. **Grupo 5 – Processo.** Formular e classificar requisitos não funcionais; comentar afirmações sobre cascata vs. iterativo/incremental.

No formato “curto” (ex. 2021) o exame é sobretudo escolha múltipla com pequenas justificações. *Atenção: respostas erradas descontam, e “pode haver mais do que uma opção correta”.*

As perguntas de “comentar a afirmação”

Quase todos os grupos terminam com “comente a afirmação...”. A afirmação costuma estar quase sempre errada de propósito. Estratégia:

- **Diga se é verdadeira ou falsa** logo na 1ª frase.
- **Justifique com a definição correta** — explique o que o diagrama realmente faz/não faz.
- **Dê o contraste.** Ex.: “o diagrama de casos de uso mostra o *que* o sistema faz, não *como* nem a ordem temporal — isso é o diagrama de sequência/atividade”.

Parte I — Matéria

1. Casos de uso: conceitos e atores

Análise de requisitos. Começa pela **visão** — documento curto, de alto nível, com as funcionalidades-chave que qualquer solução deve ter. Da visão extraem-se as **atividades principais** (casos de uso) e os **agentes externos** (atores).

Caso de uso. Situação em que o sistema é usado para satisfazer um ou mais requisitos dos seus utilizadores/stakeholders. Informalmente, é uma **pequena narrativa** de um ator a usar o sistema para atingir um objetivo. Deve:

- ser uma atividade significativa (ex.: “levantar dinheiro” é caso de uso; “introduzir o código secreto” não é);
- conter alguma interação com o sistema e produzir um resultado/benefício mensurável;
- ter um critério claro de sucesso/insucesso; não ser demasiado extenso nem trivial.

Ator. Quem faz — entidade externa (pessoa, equipamento, outro sistema) que interage com o sistema.

- **Ator primário:** interage com o sistema para atingir um objetivo (recolhe o benefício).
- **Ator secundário:** é usado pelo sistema / dá suporte (ex.: uma base de dados de credenciais, uma entidade bancária).

Stakeholder. Pessoa ou instituição com interesse nos resultados do sistema, mesmo que **não interaja** diretamente (finanças, acionistas, Governo, gestão...). Todo o ator é stakeholder, mas nem todo o stakeholder é ator.

Perguntas para descobrir atores (útil no exame)

- Quais são as principais tarefas desta pessoa no trabalho?
- Quem é responsável pela instalação/configuração do sistema?
- Que outros sistemas fornecem serviços de que precisamos?
- Quem está interessado no sistema, mesmo não interagindo com ele? (stakeholder)

NÃO servem para descobrir atores: “que interface gráfico oferecer?” ou “que protocolos usar?” (são requisitos de implementação).

2. Diagrama de casos de uso

Modela a interação **atores ↔ casos de uso**. Notação: ator = boneco (stick figure); caso de uso = elipse com o nome dentro; **fronteira do sistema** = retângulo que engloba os casos de uso (os atores ficam de fora).

Relações

Relação	Entre	Significado / quando usar	Notação
Associação	ator — caso de uso	O ator comunica/usa o caso de uso. Cada caso tem ≥ 1 ator; cada ator tem ≥ 1 caso.	linha contínua
Generalização de atores	ator — ator	O ator Y (especialização) participa em todos os casos de X e possivelmente noutros.	seta de herança (triângulo vazio)
«include»	caso — caso	Comportamento OBRIGATÓRIO e repetido extraído para reutilização. A inclui B: B faz sempre parte de A.	seta tracejada + «include»
«extend»	caso — caso	Comportamento OPCIONAL/condicional. B pode (ou não) estender A, conforme	seta tracejada + «extend»

Relação	Entre	Significado / quando usar	Notação
		condição/runtime.	
Generalização de casos	caso — caso	Um caso é variante mais específica de outro; herda comportamento e relações do caso base.	seta de herança

Include vs. Extend — a confusão clássica

- **«include» é obrigatório:** A não fica completo sem B; o comportamento de B é sempre inserido em A. Seta aponta **de A para B** (do que inclui para o incluído).
- **«extend» é opcional:** B acrescenta comportamento a A apenas em certas condições. Seta aponta **de B (extensão) para A (base)**. Ex.: “Regista conta negada” estende “Cria conta” só quando o pedido é rejeitado.

3. Descrição (especificação) de casos de uso

O diagrama mostra *que* casos existem; a **descrição textual** detalha o comportamento. Três formatos, do mais leve ao mais completo:

- **Breve:** um parágrafo único com o cenário principal de sucesso.
- **Casual:** alguns parágrafos que cobrem vários cenários (principal + alternativos).
- **Completo:** todos os passos e variantes em detalhe, com secções adicionais. Escreve-se só para a pequena percentagem de casos de maior valor/risco arquitetural.

Secções do formato completo:

Secção	Conteúdo
Nome	Começa por um verbo (ex.: “Processa venda”).
Atores primários/secundários	Quem recolhe o benefício / quem dá suporte.
Stakeholders e interesses	Quem tem interesse e que benefício quer.
Pré-condições	Situação (não trivial) que tem de se verificar antes.
Pós-condições (sucesso)	Estado do sistema após sucesso.
Fluxo de eventos	Passos do cenário principal de sucesso.
Fluxos alternativos / erros	Outras sequências e estado em caso de erro.

Os casos de uso são por natureza funcionais; os requisitos não funcionais (desempenho, capacidade, tecnologia...) registam-se na especificação de requisitos suplementar.

4. Diagramas de atividade

Especificam um **comportamento / workflow**: o caso de uso diz o *que* o sistema faz; o diagrama de atividade mostra *como* alcança o objetivo, encadeando ações de alto nível. Pertence à visão de processo. Também serve para modelar processos de negócio (detetar tarefas redundantes e “engarrafamentos”).

Elemento	Significado
Nó inicial	Círculo cheio. Início do fluxo.
Nó final de atividade	Círculo com anel. Termina toda a atividade.
Ação	Retângulo arredondado. Passo/cálculo/tarefa (ex.: Validar identidade).
Aresta	Seta entre ações (fluxo de execução ou de objetos).

Elemento	Significado
Decisão	Losango: 1 entrada, várias saídas, cada uma com condição de guarda [...]. As guardas devem ser mutuamente exclusivas.
Fusão (merge)	Losango: várias entradas, 1 saída. Termina o comportamento condicional.
Bifurcação (fork) / Junção (join)	Barra. Fork: divide em fluxos paralelos (todas as arestas percorridas). Join: só prossegue quando todas as tarefas paralelas terminam.
Partições (swimlanes / pistas)	Faixas que indicam QUEM (classe/ator) executa cada ação.
Sinais (send/receive)	Interação com participantes externos. Receive síncrono espera o sinal; send assíncrono prossegue logo.
Nó objeto / pins	Mostram o fluxo de objetos: input pin (consome) e output pin (produz).
Exceção / região de interrupção	Handler executado em vez da ação que falhou; região terminada por um evento.

Guardas mal construídas

Condições de guarda têm de ser exclusivas. Se “[item em stock]” e “[encomenda urgente]” puderem ser ambas verdadeiras ao mesmo tempo, não se sabe que aresta seguir. Reescreva combinando: [em stock & urgente], [em stock & não urgente], [sem stock]...

5. Modelo do domínio

Modelo da **análise** orientada a objetos mais clássico. Representa as **classes conceptuais** relevantes da situação real e como se relacionam — um “dicionário visual” do domínio.

Regra de ouro

O modelo do domínio descreve o PROBLEMA, não a solução. As classes são conceptuais (Venda, Cliente, Aeroporto), NÃO objetos de software com responsabilidades. Por isso NÃO se definem operações/métodos — só nome e atributos.

Análise linguística (técnica de exame):

- **Nomes/substantivos** → **candidatos a classes conceptuais** (Pessoa, Empregado, Curso).
- **Adjetivos e valores** → **candidatos a atributos** (idade, nome). Se penso na entidade como um número ou texto, é atributo; senão, é classe.
- **Verbos** → **associações** entre classes.
- **Atenção:** um atributo não deve ser de tipo complexo. Se um “atributo” é ele próprio um conceito do domínio, deve ser uma classe ligada por associação (não um atributo).

6. Diagrama de classes (design)

O diagrama UML mais conhecido: modela a estrutura estática (tipos de objetos e relações). Nas fases iniciais é conceptual; na fase de design é refinado em **classes de software** (com operações).

Conceitos OO fundamentais

- **Abstração:** ignorar detalhes irrelevantes para o contexto.
- **Herança:** classe derivada (específica) herda atributos/comportamentos da classe base (genérica).
- **Polimorfismo:** um objeto de tipo derivado pode substituir um do tipo base; permite adicionar classes sem partir a arquitetura.
- **Encapsulamento:** a classe esconde o estado interno e expõe só o necessário; pode mudar por dentro sem afetar o resto.

- **Envio de mensagens:** objetos colaboram pedindo uns aos outros para executar operações.

Visibilidade (4 níveis)

Símbolo	Nível	Acesso
+	public	Todos. Forma o interface público da classe.
#	protected	A própria classe e as classes descendentes.
~	package	Classes do mesmo pacote.
-	private	Só a própria classe. O mais restrito.

Sintaxe do atributo: [visib] nome : tipo [multiplicidade] [= default] [{propriedade}]

Sintaxe da operação: [visib] nome(par : tipo, ...) : tipoRetorno [{propriedade}]

Relações entre classes (acoplamento crescente)

Relação	Notação	Significado
Dependência	seta tracejada	Ligação fraca/breve: o cliente usa o fornecedor via parâmetro, retorno ou variável local. Uma alteração no fornecedor pode afetar o cliente.
Associação	linha contínua	Ligação mais durávoura. Pode ter nome, papéis, navegabilidade e multiplicidade.
Agregação partilhada	losango vazio	“Todo-parte” fraco: a parte pode existir sem o todo e ser partilhada por vários todos.
Agregação composta (composição)	losango cheio	“Todo-parte” forte: a parte não existe sem o todo; não partilhada (o todo controla o ciclo de vida).
Generalização (herança)	triângulo vazio	“é-um”: o tipo de um é um tipo do outro. Acoplamento mais forte.

Multiplicidade

Notação	Significado
1	exatamente um
0..1	zero ou um (opcional)
*	zero ou muitos
1..*	um ou muitos
0..2	intervalo concreto (0, 1 ou 2)

Outros elementos

- **Classe de associação:** quando a própria associação tem atributos (ex.: Inscrição entre Aluno e Disciplina com nota e data).
- **Associação n-ária:** liga 3+ classes (losango ligado por linhas); não tem navegabilidade.
- **Qualificador:** seleciona um objeto dentro de um conjunto (“chave”).
- **Restrição {xor}:** o objeto liga-se a uma associação ou a outra, não a ambas.
- **Membros estáticos:** associados à classe (uma única cópia), não a cada instância; existem durante toda a execução (ex.: contador de instâncias). Sublinhados em UML.

7. Diagrama de objetos

Mostra **instâncias concretas** num momento específico — uma “fotografia” do sistema em execução. Ajuda a validar diagramas de classes complexos.

- **Objeto:** entrada : EntradaBlog (nome sublinhado). Objeto anónimo: : Classe. Pode mostrar valores dos atributos.
- **Link:** ligação entre dois objetos; tem de corresponder a uma associação do diagrama de classes. Pode ter papéis, navegabilidade, agregação/composição.

Como validar um cenário (Grupo 3 do exame)

Para cada cenário: tente desenhar os objetos e os links. É válido apenas se RESPEITAR todas as multiplicidades e restrições do diagrama de classes.

Se uma multiplicidade obriga a “1” mas o cenário liga a 2, ou um “1..*” que ficaria a 0 — o cenário é IMPOSSÍVEL e a resposta certa é justificar a violação (sem desenhar).

8. Diagrama de sequência

Diagrama de **interação**: mostra a **ordem temporal** das mensagens trocadas entre participantes para realizar um cenário concreto. Ênfase no **tempo** (de cima para baixo).

- **Lifeline (linha de vida):** representa um único objeto. Identificação nome[seletor] : Classe (pode ser só objeto, só classe, ou ambos).
- **Barra de ativação (foco de controlo):** retângulo sobre a lifeline durante a execução de uma operação.

Mensagem	Notação	Significado
Síncrona	seta cheia preenchida	Emissor espera pela resposta.
Retorno	seta tracejada	Resposta da mensagem síncrona (opcional de desenhar).
Assíncrona	seta aberta	Emissor não espera; continua.
Criação	seta para a lifeline nova	Instancia um objeto («create»).
Destruição	X no fim da lifeline	Destrói o objeto (chama destrutor, se existir).
Auto-mensagem	seta para si própria	Objeto chama um método seu (recursivo/interno).

Fragmentos combinados (operadores)

Caixa com rótulo (pentágono) no canto; modelam controlo de fluxo. Podem aninhar-se.

Operador	Propósito
alt	Alternativas: ≥2 operandos, cada um com guarda booleana; só uma guarda pode ser true. [else] vale se nenhuma das outras for verdadeira.
opt	Interação opcional (executa se a guarda for verdadeira).
loop	Interação iterativa (com condição/limites).
break	Interação de exceção (interrompe o fragmento envolvente).
par	Interação concorrente (operandos em paralelo).
seq / strict	Ordem fraca / ordem estrita das mensagens.
critical	Interação atómica (não interrompível).
ignore / consider / assert	Filtros: parte irrelevante / relevante / obrigatória.

9. Diagrama de sequência do sistema (DSS / SSD)

Artefacto do Unified Process. Igual à notação do diagrama de sequência, mas o **sistema é visto como uma única “caixa preta”**. Mostra, para um cenário de um caso de uso, os **eventos de sistema** gerados pelos atores e a ordem em que ocorrem.

- Desenha-se para o cenário principal de sucesso e para os alternativos mais frequentes/complexos.
- Serve para descobrir que operações de sistema o software terá de oferecer em resposta aos atores — é a ponte entre a análise (caso de uso) e o design.

Diferença-chave: no DSS há só o ator e “: Sistema”. No diagrama de sequência de design abrem-se os objetos internos e as colaborações entre eles.

10. Diagrama de comunicação

Também é diagrama de interação e é **semanticamente equivalente** ao de sequência — a mesma informação, organizada de outra forma:

- **Sequência:** ênfase na ordem temporal das mensagens.
- **Comunicação:** ênfase na ligação entre participantes (é uma extensão do diagrama de objetos com mensagens).

Como se lê a ordem: pela **numeração**. Mensagens aninhadas usam numeração hierárquica: se a mensagem 1 origina outras, estas são 1.1, 1.2... Os participantes que não estiverem ligados por um link não podem comunicar.

Conexão com o Grupo 2 do exame

Para converter sequência → comunicação: mantém os mesmos objetos e mensagens, liga-os com links e numera as mensagens pela ordem em que ocorrem no diagrama de sequência (aninhando com 1.1, 1.2... onde uma mensagem despoleta outra).

11. Design OO e princípios GRASP

GRASP = **General Responsibility Assignment Software Patterns**: princípios para **atribuir responsabilidades** a classes/objetos. Um objeto tem responsabilidades de **fazer** (criar, calcular, coordenar) e de **conhecer** (dados próprios, objetos relacionados, dados deriváveis). Metodologia: *Responsibility-Driven Design* — o sistema é uma comunidade de objetos que colaboram.

Padrão	Pergunta que responde / Regra
Creator	Quem cria instâncias de A? A classe B que contém/agrega/regista/usa A de perto, ou que tem os dados para a inicializar.
Information Expert (Perito)	A quem atribuir uma responsabilidade? À classe que tem a informação necessária para a cumprir.
Low Coupling (Acopl. baixo)	Como reduzir o impacto de alterações? Atribuir responsabilidades mantendo acoplamentos desnecessários baixos.
High Cohesion (Coesão elevada)	Como manter classes focadas e manejáveis? Atribuir responsabilidades de modo que a coesão se mantenha elevada.
Controller (Controlador)	Qual o 1.º objeto, após a camada de UI, a receber um evento? Um que represente o sistema/subsistema ou o caso de uso. UI NÃO contém lógica de domínio — delega.
Polymorphism	Tratar variações por tipo via operações polimórficas, em vez de if/switch sobre o tipo.

Padrão	Pergunta que responde / Regra
Pure Fabrication (Invenção)	Classe artificial (sem correspondência no domínio) para manter coesão alta e acoplamento baixo.
Indirection (Indireção)	Objeto intermediário para desacoplar dois componentes.
Protected Variations (Variações protegidas)	Encapsular pontos de variação previsíveis atrás de um interface estável.
Bloated Controller Um controlador com demasiadas responsabilidades. Solução: adicionar mais controladores ou delegar parte das tarefas a outras classes.	

12. Padrões de desenho (GoF)

Um **design pattern** é uma solução genérica e reproduzível para um problema de design comum — um *template*, não código pronto. Os **Gang of Four (GoF)** documentaram 23, classificados por propósito. Elementos essenciais: **nome, problema, solução e consequências**.

Princípios OO subjacentes

- Encapsular o que varia.
- Preferir composição a herança; depender de interfaces/abstrações, não de implementações concretas.
- Aberto para extensão, fechado para modificação (Open/Closed).
- Uma classe deve ter uma só razão para mudar (responsabilidade única).
- “Only talk to your friends” (Lei de Demeter) e “don’t call us, we’ll call you” (inversão de controlo / injeção de dependências).

As três categorias

Categoria	Objetivo	Exemplos
Criacionais	Como os objetos são criados; tornar a instanciação mais abstrata.	Factory, Singleton, Builder, Prototype
Estruturais	Como compor classes/objetos em estruturas maiores; gerir complexidade.	Adapter, Decorator, Proxy, Bridge, Composite
Comportamentais	Como atribuir responsabilidades e desacoplar objetos; comunicação.	Strategy, Command, Iterator, Observer, Chain of Responsibility

Os quatro a saber “de cor”

- **Singleton (criacional)**: garante uma única instância com acesso global. Construtor privado, atributo estático da própria classe, método `getInstance()` que cria se ainda não existir e devolve a referência. Usos: configuração, recursos partilhados.
- **Factory (criacional)**: cria objetos omitindo os detalhes de instanciação; o tipo concreto é escolhido em runtime e referido por um interface comum (o objeto criado pertence a uma hierarquia de classes).
- **Adapter (estrutural)**: converte o interface de uma classe noutra que o cliente espera, permitindo colaborar classes com interfaces incompatíveis.
- **Observer (comportamental)**: quando o objeto observado muda de estado, todos os observadores subscritos são notificados e atualizados automaticamente. A informação pertence ao observado; os observadores subscrevem/cancelam e atualizam-se quando notificados.

13. Diagramas de componentes e de instalação

Visão de desenvolvimento (componentes). Mostra os blocos reutilizáveis do sistema. Um **componente** é uma parte modular que encapsula o seu conteúdo e pode ser substituída sem impacto negativo; define-se pelos **interface fornecido** e **interface requerido**. É um artefacto de design (sem equivalente concreto). Pode ir de poucas classes a subsistemas. Vistas: **blackbox** (só interfaces) e **whitebox** (estrutura interna, portas).

Visão física (instalação / deployment). Mostra como o software é atribuído ao hardware e como as partes comunicam. Elementos: **nó** (hardware, estereótipo «device»), **artefacto** (ficheiro/executável implantado num nó) e os **caminhos de comunicação** entre nós.

Modelo de vistas 4+1 (aparece em várias aulas)

- Vista lógica — estrutura (classes, objetos).
- Vista de processo — comportamento/concorrência (atividade).
- Vista de desenvolvimento — organização em módulos (componentes).
- Vista física — hardware (instalação).
- (+1) Vista de casos de uso — no centro, guia todas as outras.

14. Processo de desenvolvimento de software

Modelo em cascata (waterfall)

Abordagem **sequencial**: Requisitos → Design → Implementação → Verificação → Manutenção. Uma fase não começa antes de a anterior terminar; o cliente sai de cena após a fase inicial.

Pressupostos (irrealistas): os requisitos são conhecidos no início e não mudam; o cliente sabe exatamente o que quer; o design pode ser feito de forma abstrata sem erro; a tecnologia mantém-se estável. Limita a capacidade de reagir a mudanças.

Pergunta clássica de exame

“O modelo em cascata define um processo incremental.” → FALSO. O cascata é sequencial e de entrega única no fim; não é incremental nem iterativo. Incremental/iterativo são os métodos ágeis e o Unified Process.

Métodos ágeis

- Desenvolvimento iterativo e incremental, iterações curtas, planeamento adaptativo, entregas incrementais.
- Cliente incluído em todo o processo; simplicidade, comunicação, equipas auto-organizadas; resposta rápida à mudança (com base em feedback).

Unified Process (UP) / RUP

Processo **iterativo, incremental e guiado pelo risco** para sistemas OO. Iterações iniciais com maior desvio; por feedback e adaptação o sistema converge para os requisitos e o design adequados. RUP é configurável — nenhum processo serve todos os projetos.

Quatro fases (eixo dinâmico):

Fase	Foco / marco (milestone)
Conceção (Inception)	Visão, âmbito, viabilidade, análise de risco/custo. Decidir se o projeto avança.
Elaboração	Domínio do problema, análise e arquitetura estável; mitigar os maiores riscos.
Construção	Desenvolver e testar os componentes; alcançar versões úteis (alfa/beta).

Fase	Foco / marco (milestone)
	Produto pronto a entregar.
Transição	Beta, conversão de dados, treino de utilizadores, deployment. Cliente satisfeito e sistema autónomo.

Disciplinas/workflows (eixo estático): Modelação de negócio, Requisitos, Análise e Design, Implementação, Teste, Deployment — mais as de suporte (Gestão de projeto, Configuração/mudança, Ambiente). Cada iteração termina com uma *release*.

Parte II — Exercícios resolvidos

Os exercícios seguem o formato real dos exames de MeD. Em cada um indico o raciocínio, não só a resposta — é isso que dá pontos.

Exercício 1 — Casos de uso (Grupo 1)

Enunciado

Uma empresa de restauração quer uma app móvel onde os clientes consultam o menu, se registam e fazem pedidos de entrega ao domicílio. A escolha e o pagamento só são possíveis após autenticação; o pagamento é validado por uma entidade bancária. Os funcionários de entrega, após autenticação, consultam e efetuam entregas.

a) Identifique casos de uso e atores. b) Atualize: “O pagamento pode ser feito ao funcionário na entrega; nesse caso o registo do pagamento é feito pelo funcionário juntamente com o registo de entrega.” c) Comente: “Um diagrama de casos de uso também pode representar restrições aos serviços disponibilizados.”

Resolução

a) Atores e casos de uso.

- **Atores primários:** Cliente, Funcionário de entrega.
- **Ator secundário:** Entidade Bancária (dá suporte — valida o pagamento; não recolhe benefício).
- **Casos de uso:** Consultar menu (Cliente, sem autenticação); Registar; Autenticar; Efetuar pedido e Efetuar pagamento (Cliente autenticado); Consultar entregas e Efetuar entrega (Funcionário).

Relações: “Efetuar pagamento” faz «include» de “Validar pagamento” (com a Entidade Bancária) — validação obrigatória. “Efetuar pedido” e “Efetuar pagamento” dependem de “Autenticar” (pré-condição; modela-se com «include» de Autenticar, ou como pré-condição na descrição).

b) Atualização. Acrescentar caso “Registar pagamento na entrega” associado ao Funcionário. Como o pagamento na entrega só acontece **às vezes**, é comportamento opcional: liga-se a “Efetuar entrega” com «extend» (a extensão aponta para o caso base “Efetuar entrega”).

c) Comentário. Verdadeira (parcialmente). O diagrama foca-se em o *que* o sistema faz e em quem o usa, mas permite exprimir restrições de acesso através das associações ator-caso (só quem está ligado pode usar) e da generalização de atores. Detalhes finos de restrições (pré/pós-condições) ficam, no entanto, na descrição textual do caso de uso, não no diagrama.

Exercício 2 — Atividade → Sequência → Comunicação (Grupo 2)

Enunciado

Diagrama de atividades: “Efetua Pedido” → “Verifica Stock”; se [insuficiente] → “Apresenta Mensagem”; se [outro] → “Atualiza Stock”.

Métodos: “Efetua Pedido” = pedido() e “Apresenta Mensagem” = mensagem() na classe Vista; “Verifica Stock” = verifica() e “Atualiza Stock” = atualiza() na classe Modelo.

a) Diagrama de sequência, assumindo que um objeto Controlador envia as mensagens via efetuaEncomenda(). b) Diagrama de comunicação equivalente. c) Comente: “Um diagrama de atividades não permite exibir as alterações de estado de um objeto durante as atividades.”

Resolução

a) Diagrama de sequência. Participantes: : Vista, : Controlador, : Modelo. O ator inicia em pedido() na Vista, que delega no Controlador (efetuaEncomenda()), que orquestra o Modelo. Sequência:

6. Vista → Controlador : efetuaEncomenda()
7. Controlador → Modelo : verifica() (→ retorna disponibilidade)
8. Fragmento **alt** sobre o resultado de verifica():
 - **[stock insuficiente]** Controlador → Vista : mensagem()
 - **[else / outro]** Controlador → Modelo : atualiza()

Note a aplicação de GRASP Controlador: a Vista (UI) não contém lógica — delega no Controlador, que coordena o Modelo. A decisão do diagrama de atividade vira um fragmento alt.

b) Diagrama de comunicação. Mesmos objetos ligados por links, com as mensagens numeradas pela ordem da sequência:

- 1: efetuaEncomenda() Vista → Controlador
- 1.1: verifica() Controlador → Modelo
- 1.2 [insuficiente]: mensagem() Controlador → Vista
- 1.3 [outro]: atualiza() Controlador → Modelo

As mensagens despoletadas por efetuaEncomenda() são aninhadas (1.1, 1.2, 1.3). Os links têm de existir entre pares que comunicam (Vista–Controlador e Controlador–Modelo).

c) Comentário. Falsa. O diagrama de atividade pode mostrar alterações de estado de um objeto através do **fluxo de objetos** (nós objeto com o estado entre parêntesis, ex.: pedido[criado] → pedido[validado]). Não é o seu foco principal, mas é possível.

Exercício 3 — Classes → Objetos (Grupo 3)

Enunciado

Diagrama de classes: Pessoa (atributo +autor) ; Cliente. Recurso liga-se a Pessoa-autor (1 Recurso — 1..* autor) e a Requisição (Recurso * — 0..1 Requisição). Recurso tem duas subclasses por generalização: Colectânea e Livro. Colectânea inclui Recursos (0..1 Colectânea — 0..2 Recurso). Editor (1) edita 1..* Recurso (um Recurso tem 0..1 Editor).

Diga se cada cenário é representável; se sim, desenhe o diagrama de objetos, se não, justifique:

- O Luís requisitou o livro “Introdução ao UML”, autor António, sem colectânea, editor “Nova Editora”.
- Uma colectânea inclui outra colectânea de todos os livros do mesmo autor e um livro com a biografia desse autor; a colectânea não tem editor.

Resolução

a) **Possível.** Diagrama de objetos:

- luis : Cliente — link → r1 : Requisição
- r1 : Requisição — link → uml : Livro (1 recurso na requisição; respeita Recurso * — 0..1 Requisição)
- uml : Livro {título=“Introdução ao UML”} — autor → antonio : Pessoa (1..* autor: 1 ✓)
- uml : Livro — editor → nova : Editor (0..1 editor: 1 ✓). Sem link a Colectânea (0..1 ✓).

Verificação: todas as multiplicidades obrigatórias estão satisfeitas e nenhuma é excedida → válido.

b) **Avaliar a multiplicidade Colectânea — Recurso (0..2).** A colectânea exterior inclui: (1) outra colectânea + (2) um livro = **2 recursos**, o que cabe em 0..2 ✓. A colectânea interior inclui “todos os livros do mesmo autor” — se forem mais de 2, **viola o limite 0..2** e o cenário torna-se **impossível**; se forem ≤2, é representável. A colectânea sem editor respeita 0..1.

Lição

Num cenário com “todos os ...”, confirme sempre se o número implicado cabe na multiplicidade máxima. É aí que está a armadilha.

Exercício 4 — Da análise ao design (Grupo 4)

Enunciado

Requisito: “O cliente registado cria um pedido de uma refeição para entrega ao domicílio, seleccionando os pratos do menu e indicando o local de entrega. Todos os funcionários de entrega recebem uma notificação.”

a) Fluxo de eventos do caso “Efetua Pedido” (cenário principal). b) Modelo do domínio (análise linguística). c) DSS para “Efetua Pedido”. d) Desenho de UMA das mensagens, com padrão MVC.

Resolução

a) Fluxo de eventos (cenário principal de “Efetua Pedido”):

9. O Cliente inicia um novo pedido.
10. O sistema apresenta o menu de pratos disponíveis.
11. O Cliente seleciona os pratos (repete até terminar).
12. O Cliente indica o local de entrega.
13. O sistema regista o pedido e notifica os funcionários de entrega.

b) Modelo do domínio (análise linguística). Substantivos → classes; relações → associações:

- **Classes:** Cliente, Pedido, Prato, Menu, Funcionário (de entrega), LocalDeEntrega.
- **Associações:** Cliente 1 — * Pedido; Pedido 1..* — * Prato; Menu 1 — * Prato; Pedido * — 1 LocalDeEntrega.
- **Atributos (não operações!):** Cliente.nome; Prato.nome, Prato.preço; Pedido.data; LocalDeEntrega.morada.

c) DSS (sistema como caixa preta). Eventos de sistema gerados pelo Cliente:

- `iniciarPedido()`
- `selecionarPrato(prato)` (em loop)
- `indicarLocal(morada)`
- `confirmarPedido()` → o : Sistema responde com confirmação.

d) Desenho (design) de `confirmarPedido()` com MVC + GRASP. A mensagem da Vista vai para um Controlador (não põe lógica na UI — GRASP Controlador) que coordena o Modelo:

- 1: `confirmarPedido()` Vista → ControladorPedido
- 1.1: `create()` ControladorPedido → Pedido (GRASP Creator — quem agrega/regista o Pedido cria-o)
- 1.2: `registar(pedido)` ControladorPedido → Modelo
- 1.3: `notificar()` ControladorPedido → Funcionário (cada um) — ou via padrão Observer, se os funcionários subscreverem novos pedidos.

Exercício 5 — Processo de desenvolvimento (Grupo 5)

Enunciado

Vai gerir o desenvolvimento da app de restauração usando o modelo em cascata.

a) Formule e classifique dois requisitos não funcionais. b) Comente: “O modelo em cascata define um processo de desenvolvimento incremental.”

Resolução

a) Requisitos não funcionais (RNF) — formular + classificar:

- **Desempenho:** “O sistema deve apresentar o menu em menos de 2 segundos com 1000 utilizadores em simultâneo.”
- **Segurança:** “Os dados de pagamento devem ser transmitidos cifrados (TLS) e nunca armazenados em claro.”

Outras categorias válidas: usabilidade, fiabilidade/disponibilidade, portabilidade, manutenibilidade, capacidade.

b) Comentário. Falsa. O modelo em cascata é **sequencial e de entrega única**: as fases ocorrem uma após a outra e o produto só é entregue no fim. Não produz incrementos sucessivos do produto ao longo do tempo nem itera — isso caracteriza os métodos ágeis e o Unified Process. O cascata pressupõe ainda que os requisitos são conhecidos e estáveis desde o início, o que limita a resposta à mudança.

Parte III — Checklist final e erros a evitar

Confusões mais penalizadas

Não confundir	Memória rápida
include vs. extend	include = obrigatório (A→B); extend = opcional (extensão→base).
Agregação vs. composição	Losango vazio = parte sobrevive ao todo; losango cheio = parte morre com o todo.
Modelo de domínio vs. classes de design	Domínio = sem operações (problema); design = com operações (solução).
Sequência vs. comunicação	Equivalentes; um enfatiza o tempo, o outro as ligações. Comunicação usa numeração.
DSS vs. sequência de design	DSS = caixa preta (ator + Sistema); design = objetos internos abertos.
Casos de uso «include» (UML) vs. herança OO	São coisas diferentes; não misturar terminologia.
Cascata vs. iterativo	Cascata = sequencial, entrega única; ágil/UP = iterativo e incremental.

Antes de entregar, verifico que...

- Cada caso de uso tem ≥ 1 ator e cada ator ≥ 1 caso; atores fora da fronteira do sistema.
- As setas de include/extend apontam no sentido certo e têm o estereótipo escrito.
- As guardas de decisão (atividade) e de alt (sequência) são mutuamente exclusivas e cobrem todos os casos ([else]).
- No modelo do domínio NÃO escrevi operações; conceitos complexos são classes (associação), não atributos.
- Os diagramas de objetos respeitam TODAS as multiplicidades e restrições do diagrama de classes.
- A UI delega no Controlador (GRASP) e a coesão/acoplamento foram considerados no design.
- Nas perguntas de “comentar”: disse V/F na 1ª frase + justifiquei com a definição correta + dei o contraste.
- No formato de escolha múltipla: lembrei-me de que há desconto por resposta errada e que pode haver mais do que uma opção certa.

Bom estudo. Domina os cinco grupos do exame e a maior parte da cotação está garantida.